



Cisco Secure Access RA VPN Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-30

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3
Reference documentation	3

Overview

The Crogl Cisco Secure Access RA VPN connector reads remote-access VPN (AnyConnect / Secure Client) data — historical connect/disconnect/failure events and currently active sessions — from a customer's Cisco Secure Access organization. Authentication is a single **Secure Access API key** (a client ID + client secret pair), which Crogl exchanges server-side for a short-lived bearer token via OAuth2 client-credentials. There is no separate app registration; the API key itself is the credential object, created directly in the Secure Access dashboard.

Generating the key requires the **Full Admin** user role in Secure Access — whoever creates it needs that role assigned in **Admin > Accounts** first. If you're an FDE walking a customer through this live, expect their Secure Access administrator to drive the console while you confirm the resulting values.

NOTE

Secure Access documents two informal "kinds" of API key by what scopes are attached to them, not a type selector in the creation form itself: a key with no `admin.apikeys:*` scopes checked is a plain business-data key (what this connector needs); a key that also has `admin.apikeys:*` scopes checked can create/read/update/delete **other** API keys (a "Key Admin" key). Do not check any API-key-administration resource when creating this connector's key — it only needs the two read-only resources listed below.

What kind of account to create

Not a per-user account or app registration — a **Secure Access API key**, created from **Admin > API Keys** in the Secure Access dashboard by a user with the Full Admin role. Each key is independently scoped at creation time to specific key-scope/resource pairs, each grantable as Read-Only or Read/Write.

Permissions to grant

This connector is read-only and needs exactly two resources, both set to **Read-Only**:

- Key scope **Admin**, resource **VPN** — maps to the `admin.vpn:read` API scope; required for the connector's `vpn-sessions` query (currently active VPN sessions).
- Key scope **Reports**, resource **Aggregations** — maps to the `reports.aggregations:read` API scope; required for the connector's `remote-access-events` query (historical connect/disconnect/failure events).

IMPORTANT

The on-screen resource labels above ("VPN" under Admin, "Aggregations" under Reports) are inferred from the API's own scope-name convention (`admin.vpn:read`, `reports.aggregations:read`), not confirmed against a live checkbox tree while writing this guide — the console may show a slightly different display label for either resource. If the exact label isn't obvious in the scope tree, look for the resource whose name most closely matches "VPN" under the **Admin** key scope, and "Aggregations" under the **Reports**

key scope, and verify against the customer's console.

Do not check any other resource under either key scope, and do not check any resource under the API-key-administration surface — the narrowest working grant is the two Read-Only checks above.

There is no separate admin-consent step as in an OAuth authorization-code flow; the only gate is that the person creating the key must themselves hold the Full Admin role.

Generating the credential

Per Cisco's own Secure Access documentation ("Add an API Key"):

1. Sign in to the Secure Access dashboard as a Full Admin user.
2. Navigate to **Admin > API Keys**.
3. Click **API Keys**, then click **Add**.
4. Enter a **name** for the key (fewer than 256 characters) and, optionally, a **description**.
5. Under **key scopes**, expand **Admin** and check the **VPN** resource; expand **Reports** and check the **Aggregations** resource (see [Permissions to grant](#) above for the caveat on exact labels).
6. For each checked resource, choose **Read-Only** (not Read/Write).
7. For **Expiry Date**, choose an expiration date or **Never expire**. If the customer's security policy requires periodic rotation, set one; otherwise no expiration is simplest to operate.
8. **(Optional)** Under **Network Restrictions**, add up to ten public IP addresses or CIDR blocks if the customer wants to restrict which source networks may use this key, then click **Add**.
9. Click **Create Key**.
10. **Copy and save the displayed API Key and Key Secret immediately** — Secure Access shows the Key Secret exactly once and cannot redisplay or recover it. If it's lost before you copy it, delete the key and create a new one rather than trying to retrieve the value.

Mapping values into Crogl

Two values map to `community/cisco-secure-access-ravpn/connector.json`'s `'credential_schema.fields` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>api_key</code>	The API Key (client ID) value shown once at key creation	Not secret by itself, but treat it as sensitive — it identifies the credential.

Crogl field	Where it came from	Notes
<code>api_secret</code>	The Key Secret value shown once at key creation, alongside the API Key	Treat as a password. Never paste it into chat, a ticket, or this document. Crogl sends it as part of the OAuth2 client-credentials exchange (HTTP Basic <code>key:secret</code> to the token endpoint) automatically once entered — it is never sent as a request header directly.

Verifying the setup

Right after creating the connector instance in Crogl, confirm the setup:

- **Success:** a query against either resource (e.g. asking Crogl to "list active VPN sessions" or "show remote-access VPN events from the last day") returns rows, or a clean empty result if there's genuinely no matching data. An empty result is a valid, successful response, not an error.
- **Token-mint failure (fails before ever reaching Secure Access's own API):** the `api_key` / `api_secret` pair is wrong, was truncated when copied, or the key was deleted/regenerated since it was entered into Crogl. Generate a new key (the old secret cannot be recovered) and re-enter both `api_key` and `api_secret`.
- **403 on vpn-sessions specifically:** the key authenticated fine, but the **Admin** > **VPN** resource wasn't checked (or wasn't checked as Read-Only) at key-creation time. Go back into **Admin** > **API Keys**, open the key, and confirm that resource is granted.
- **403 on remote-access-events specifically:** same root cause, but for the **Reports** > **Aggregations** resource instead — confirm that grant separately, since the two scopes are independent and one can be correctly granted while the other is missing.
- **400 on remote-access-events:** almost always a malformed or missing `from/to` time window, or a window wider than the documented 30-day maximum — not a credential problem. Re-check the query parameters before assuming the key is misconfigured.
- **429:** rate limited, not a credential problem — the same key is valid, just making requests faster than Secure Access allows (5 requests/second per key for both scopes this connector uses). Back off and retry rather than regenerating the key.

Reference documentation

Every claim in this guide and in the connector's `SKILL.md` (endpoints, scopes, field schemas, rate limits, the key-creation procedure) was checked against these Cisco pages directly, not recalled from training data:

- [Cisco Secure Access API, Authentication](#) — OAuth2 client-credentials model, key types.
- [Cisco Secure Access API, Getting Started](#) — token-request example (`curl --user '<key>:<secret>'`), confirming HTTP Basic auth to the token endpoint.

- [Cisco Secure Access VPN User Connections API, Overview](#) — the `admin.vpn:read/admin.vpn:write` scope and `vpn-sessions` resource this connector reads.
- [List VPN Connections](#) — the OpenAPI spec (path, query parameters, response schema) backing this connector's `vpn-sessions` resource.
- [Get Remote Access Events](#) — the OpenAPI spec backing this connector's `remote-access-events` resource, including the `anyconnectversion/connectionevent` fields.
- [Cisco Secure Access Reporting API](#) — the `reports.aggregations:read` scope and the documented 30-day maximum `from/to` window.
- [Cisco Secure Access API Rate Limits](#) — the per-scope rate limits cited in Verifying the setup above.
- [Create API Key \(Key Admin API\)](#) — confirms the Standard-vs-Key-Admin distinction is a matter of which scopes are attached to a key, not a separate type selector.
- [Add an API Key](#) — the exact console procedure (Admin > API Keys > Add) this guide's Generating the credential section is based on.